

Codex 实现任务：Windows 极简家具自动抠图工具

请直接完成一个可运行的 Windows 本地桌面工具，不要只输出示例、伪代码或架构建议。完成代码、依赖文件、安装脚本、启动脚本、README 和基本测试。

不要反复向我确认需求。遇到非关键细节时自行采用最稳妥、最简洁的实现。

一、项目目标

开发一个运行于 Windows 10/11 的极简家具抠图工具。

用户打开一张实拍家具照片后，程序使用高质量本地模型自动识别家具主体并去除背景，输出透明背景 PNG。

正常情况下整个流程只有：

打开图片
→ 自动抠图
→ 查看结果
→ 保存透明 PNG

只有自动抠图识别错家具、保留了其他物体或漏掉主要家具时，用户才需要进入“框选重算”模式：

点击框选重算
→ 鼠标框住正确的家具
→ 模型只对框选区域重新抠图
→ 查看结果
→ 保存透明 PNG

工具不需要复杂人工修图功能。

输出透明 PNG 后，用户会把图片交给 image-2 模型生成真实室内效果图，因此目标是：

- 家具主体整体正确；
- 外形、比例、颜色和纹理保持原图；
- 不要保留明显的大块背景；
- 不要求达到专业商业修图级发丝边缘精度；
- 少量边缘瑕疵可以接受；
- 不允许对家具主体进行生成式重绘。

二、目标运行环境

操作系统: Windows 10 / Windows 11
处理器: Intel Core i5-12400
内存: 32GB
独立显卡: 无
运行方式: 纯 CPU

重要约束:

- 不依赖 NVIDIA 显卡;
- 不依赖 CUDA;
- 不使用 FP16 CPU 推理;
- 使用 float32;
- 用户不在乎推理速度;
- 质量优先于速度;
- 允许单张图片处理较长时间;
- 模型推理期间 GUI 不得卡死或无响应。

三、核心模型

默认且主要使用:

ZhengPeng7/BiRefNet_HR-matting

用途:

- 自动识别家具主体;
- 生成带有软边过渡的 Alpha 蒙版;
- 用于整图自动抠图;
- 用于用户框选后的局部重新抠图。

优先使用官方 Hugging Face 权重和官方 PyTorch 推理方式。

加载方式可参考:

```
from transformers import AutoModelForImageSegmentation

model = AutoModelForImageSegmentation.from_pretrained(
    "ZhengPeng7/BiRefNet_HR-matting",
    trust_remote_code=True
)
```

必须核对并遵循 BiRefNet 官方仓库或 Hugging Face 模型页的最新图像预处理方式, 不要自己凭空编造归一化参数和输出解析方式。

默认设备：

```
device = torch.device("cpu")
model.to(device)
model.eval()
```

推理必须使用：

```
with torch.inference_mode():
    ...
```

不要加入以下模型：

- rembg
- U2Net
- SAM
- SAM2
- SAM3
- MobileSAM
- ViTMatte
- 其他生成式模型

第一版只围绕 BiRefNet_HR-matting 做好。

四、极简界面

使用：

```
Python
PySide6
PyTorch CPU
Transformers
Pillow
NumPy
OpenCV
```

主界面只保留必要功能。

4.1 顶部工具栏

只需要以下按钮：

```
打开图片
自动抠图
```

框选重算
取消框选
保存 PNG
重置
设置

4.2 中央显示区域

采用左右对比布局：

左侧：原始照片
右侧：透明背景结果

右侧默认使用棋盘格背景显示透明区域。

增加三个简单预览背景按钮：

棋盘格
白色
黑色

用途是快速判断是否仍然保留明显背景或出现严重边缘问题。

4.3 底部状态栏

显示：

当前状态
模型加载状态
正在处理
处理完成
处理失败
当前图片尺寸
输出图片尺寸
推理耗时

处理时显示不确定进度条或旋转等待动画。

五、标准自动抠图流程

打开图片后，不必自动立即推理，避免误打开大图时无法取消。

用户点击“自动抠图”后执行以下流程：

读取原始图片
→ 修正 EXIF 旋转方向
→ 转换为 RGB
→ 按官方方式预处理
→ BiRefNet_HR-matting CPU 推理
→ 提取连续 Alpha 蒙版
→ Alpha 映射回原始图片尺寸
→ 原始 RGB 与 Alpha 合成为 RGBA
→ 显示透明结果

必须保持：

输出宽度 = 原图宽度
输出高度 = 原图高度

不得把用户原始图片永久缩小到 2048×2048 后再导出。

模型可以在 2048×2048 或模型官方指定尺寸上推理，但最终 Alpha 必须高质量映射回原图尺寸，再与原始 RGB 合成。

不得让模型修改原始家具 RGB。

最终输出必须是：

原始家具 RGB
+
模型生成的 Alpha

六、输入图像处理

支持：

.jpg
.jpeg
.png
.webp

打开图片时必须：

1. 应用 EXIF Orientation;
2. 转换到标准 sRGB/RGB;
3. 保留原始尺寸;
4. 建立预览图;

5. 不直接修改原文件。

模型输入缩放时：

- 保持长宽比例；
- 禁止直接拉伸导致家具变形；
- 使用 letterbox、padding 或官方推荐方式；
- 必须记录缩放比例和 padding；
- 输出蒙版时准确去除 padding；
- 再映射到原图坐标系。

如果官方 BiRefNet 推理代码要求固定正方形输入，可以在模型层内部处理，但输出不能发生比例偏移。

七、框选重算功能

这是本工具唯一需要用户手动介入的功能。

7.1 用户操作

用户点击“框选重算”后：

1. 鼠标在左侧原图上拖动矩形；
2. 矩形区域半透明显示；
3. 松开鼠标后显示确认状态；
4. 自动对框选区域重新运行 BiRefNet_HR-matting；
5. 将结果映射回原始图片；
6. 框外区域全部设为透明；
7. 显示新的结果。

不需要弹出复杂对话框。

7.2 框选区域处理

用户画出的矩形不能直接作为最终裁剪边界。

程序应自动增加少量 padding，防止用户框得太紧而切掉家具边缘。

默认：

padding = 框选宽高较大值的 5%

允许在设置中调整：

0%
5%

10%
15%

增加 padding 后不能超出原图范围。

处理流程：

用户框选区域
→ 自动向四周扩展 padding
→ 从原始图片裁剪 ROI
→ 对 ROI 运行 BiRefNet_HR-matting
→ 得到 ROI Alpha
→ 将 ROI Alpha 映射回原始坐标
→ 框选扩展区域外 Alpha 全部设为 0
→ 与原始 RGB 合成最终 RGBA

框选后不要继续使用第一次整图推理的蒙版。

框选重算应当完全以原始照片的 ROI 为输入，重新生成结果。

7.3 小框保护

如果用户框选区域过小，例如：

宽度小于 50 像素
或
高度小于 50 像素

则不执行推理，并提示：

框选区域太小，请重新框选家具主体。

八、Alpha 蒙版处理

BiRefNet_HR-matting 输出应保持连续 Alpha，不要默认二值化。

禁止直接执行：

$\alpha = \alpha > 0.5$

Alpha 应保持：

0.0 ~ 1.0

只允许做最基础且不会明显改变主体的处理：

1. 限制到 0~1；
2. 消除 NaN 和 Inf；
3. 映射为 0~255；
4. 高质量缩放到原始尺寸。

不要默认加入：

- 腐蚀；
- 膨胀；
- 羽化；
- 锐化；
- 边缘重建；
- 去白边；
- 去黑边；
- 人工阈值；
- 自动填孔；
- 自动删除小区域。

因为这些后处理可能错误删除家具细腿、镂空、软包边缘或床品。

可以在设置中提供一个默认关闭的选项：

清除极小背景杂点

但第一版不强制实现。

九、输出规则

保存格式：

PNG RGBA
8-bit Alpha
Straight Alpha
非预乘 Alpha

默认输出文件名：

原文件名_cutout.png

例如：


```
IMG_20260623_123456.jpg
→
IMG_20260623_123456_cutout.png
```

保存时：

- 默认保存到原图所在目录；
- 允许用户另选路径；
- 不覆盖原始文件；
- 输出尺寸与原图一致；
- 输出 RGB 来自原始图片；
- 完全透明区域可以将 RGB 清零；
- 半透明边缘区域必须保留原始 RGB。

同时在设置中提供一个可选项：

同时保存 Alpha 蒙版

启用后额外输出：

原文件名_alpha.png

但默认关闭，避免界面和文件过于复杂。

十、状态管理

程序内部只需要保存：

```
original_image
preview_image
current_alpha
current_rgba
selected_box
current_file_path
output_file_path
```

不需要：

- 图层系统；
- 操作历史树；
- 撤销重做系统；
- 项目工程文件；
- 多蒙版系统；
- 手工画笔数据；

- 套索数据。

“重置”按钮执行：

清除当前 Alpha
清除当前结果
清除框选
恢复到刚打开原图的状态

如果自动整图结果不满意，用户可以直接框选重算，不需要先重置。

十一、线程和响应性

模型加载和推理必须在后台线程执行。

推荐：

QThread
或
QThreadPool + QRunnable

不得在 Qt 主线程执行模型加载或推理。

处理时：

- 禁用“自动抠图”和“框选重算”按钮；
- 保留窗口移动、最小化和界面刷新；
- 显示处理中状态；
- 推理完成后恢复按钮；
- 推理失败后显示可理解的错误信息。

禁止只在控制台输出错误。

错误对话框至少应显示：

模型加载失败
图片读取失败
内存不足
推理失败
模型下载失败
保存失败

同时将完整异常写入：

```
logs/app.log
```

十二、CPU 优化原则

用户不追求速度，但程序必须合理使用 CPU 和内存。

使用：

```
torch.set_num_threads(...)
torch.set_num_interop_threads(...)
```

线程数不要盲目写死为 12。

建议根据逻辑核心自动决定，例如：

```
cpu_count = os.cpu_count() or 6
torch.set_num_threads(max(1, cpu_count - 2))
```

模型只加载一次并保持在内存中。

不要每处理一次图片就重新创建模型。

禁止：

- CPU 上使用 float16；
- 调用 torch.cuda；
- 检测不到 CUDA 就报错；
- 强制安装 CUDA 版 PyTorch；
- 同时加载多个大型模型；
- 把原始 4K/8K RGB 图片复制很多份。

完成一张图片后，应释放不再使用的中间张量。

十三、模型下载和本地缓存

首次运行时，如果本地没有模型：

1. 提示需要下载模型；
2. 用户确认后下载；
3. 显示下载状态；
4. 下载后缓存在本地；
5. 后续可完全离线运行。

默认模型：

ZhengPeng7/BiRefNet_HR-matting

支持环境变量和 Hugging Face 标准缓存。

程序设置中提供：

模型缓存目录

并允许用户选择已经下载好的本地模型文件夹。

如果 Hugging Face 自动下载失败，应提示：

无法下载模型。请检查网络、代理或手动选择本地模型目录。

不要让程序直接退出。

十四、项目结构

建议结构：

```
furniture_cutout/
├─ app.py
├─ requirements.txt
├─ install.bat
├─ start.bat
├─ README.md
├─ config.json
├─ assets/
│   └─ app_icon.ico
├─ logs/
├─ furniture_cutout/
│   ├── __init__.py
│   ├── main_window.py
│   ├── image_view.py
│   ├── box_selector.py
│   ├── inference_worker.py
│   ├── birefnet_engine.py
│   ├── image_processing.py
│   ├── exporter.py
│   ├── settings.py
│   └─ logging_config.py
└─ tests/
```

```
├─ test_image_mapping.py
├─ test_box_coordinates.py
├─ test_export_size.py
└─ test_alpha_composite.py
```

不要过度拆分。

不要引入：

- 数据库；
- Web 服务；
- Flask；
- FastAPI；
- Gradio；
- Electron；
- Docker；
- 用户账户；
- 云端上传；
- 遥测；
- 自动更新系统。

十五、Windows 安装和启动

提供：

```
install.bat
start.bat
```

`install.bat` 应完成：

1. 检查 Python；
2. 创建 `.venv`；
3. 激活虚拟环境；
4. 安装 CPU 版 PyTorch；
5. 安装 requirements；
6. 给出明确的成功或错误提示。

不要安装 CUDA 版 PyTorch。

`start.bat` 应：

1. 激活 `.venv`；
2. 启动 `app.py`；
3. 发生异常时保持窗口，方便查看错误；
4. 日志同时写入 `logs/app.log`。

推荐 Python 版本：

Python 3.11 64-bit

请锁定一组互相兼容的依赖版本，并在 README 中解释如何重新安装。

十六、设置界面

设置界面保持极简，只需要：

模型缓存目录
框选自动扩展比例
CPU 线程数
默认输出目录
同时保存 Alpha 蒙版
首次打开图片后是否自动开始抠图

默认配置：

模型：BiRefNet_HR-matting
框选扩展：5%
CPU 线程：自动
输出目录：原图片目录
保存 Alpha：关闭
打开后自动抠图：关闭

不要把大量模型参数暴露给普通用户。

十七、必须验证的功能

完成代码后，至少验证：

1. JPG 可以正常打开；
2. PNG 可以正常打开；
3. WebP 可以正常打开；
4. EXIF 旋转的手机照片方向正确；
5. 自动抠图输出尺寸与原图一致；
6. 输出 PNG 具有透明通道；
7. 原始家具 RGB 没有被重新生成；
8. 框选坐标在缩放和拖动画布后仍然准确；
9. 框选 ROI 映射回原图位置正确；
10. 框外区域完全透明；
11. 框选 padding 不超出图片边界；
12. 保存不会覆盖原图；
13. 模型推理期间界面可以正常刷新；

- 14. 无 CUDA 的电脑可以启动和推理；
- 15. 重复处理图片不会重复加载模型；
- 16. 处理失败后仍然可以打开下一张图片；
- 17. 4K 手机照片可以完成输出；
- 18. 保存后的 PNG 可以被常见图片软件正常打开。

十八、验收标准

工具最终必须达到以下体验：

双击 start.bat
→ 打开桌面程序
→ 拖入或打开家具照片
→ 点击自动抠图
→ 查看透明结果
→ 正确则点击保存 PNG

如果自动结果识别错主体：

点击框选重算
→ 框住正确家具
→ 等待重新计算
→ 点击保存 PNG

整个主界面不应出现任何不必要的高级功能。

不实现：

- 手工画笔；
- 橡皮擦；
- 套索；
- 魔棒；
- 边缘精修；
- ViTMatte；
- SAM；
- 多模型对比；
- 批量处理；
- PSD；
- 图层；
- 项目文件；
- 自动生成室内场景。

最终交付必须包括：

完整源码
requirements.txt

```
install.bat
start.bat
README.md
基础测试
清晰的错误处理
```

请在实现完成后自行运行测试并检查真实启动流程，不要仅以“代码能够导入”作为完成标准。

十九、实现优先级

按以下顺序完成，但最终一次性交付全部内容：

1. BiRefNet_HR-matting CPU 推理
2. 原尺寸 Alpha 映射
3. RGBA PNG 输出
4. PySide6 主界面
5. 后台推理线程
6. 框选 ROI 重算
7. Windows 安装与启动脚本
8. 错误处理和日志
9. 测试
10. README

不要在核心流程完成前优化视觉动画或制作复杂安装包。

最终目标不是做一个通用图片编辑器，而是做一个：

```
自动抠家具
识别错了就框一下
然后保存透明 PNG
```

的极简 Windows 工具。